

COMPSYS302: Python

8. Class

Ho Seok AHN

hs.ahn@auckland.ac.nz



**THE UNIVERSITY
OF AUCKLAND**
FACULTY OF ENGINEERING

8.1 The class Statement

□ The class Statement

- Python is an object-oriented programming language like C++ and Java.
- The **class** statement is the most common way to create a class object.
- **class** is a single-clause compound statement with the following syntax:

```
class classname [(base-classes)]:  
    statement(s)
```

- 'classname' is an identifier. 'base-classes' is an optional comma-delimited series of expressions whose values must be class object.
- The body of a class is where you normally specify the attributes of the class.

- Ex) 8_01_Class.py

```
class Person:  
    Name = "default"  
  
p1 = Person()  
p2 = Person()  
  
print ("Name of p1 instance: " + p1.Name)  
print ("Name of p2 instance: " + p2.Name)
```

(result)

Name of p1 instance: default

Name of p2 instance: default

8.2 Function Definition in Class

□ Function Definition in a Class body

- Most class bodies include def statements, as functions are important attributes for class objects.
- A method defined in a class body always has a mandatory first parameter, named **'self'**, that refers to the instance on which you call the method.
- Here is an example of a class that includes a method definition:

```
class classname [(base-classes)]:  
    def functionname(self):  
        statement(s)
```

- Ex) 8_02_ClassFunction.py

```
class Person:  
    Name = "default"  
    def PrintName(self):  
        print ("My name is " + self.Name)  
p1 = Person(); p2 = Person()  
p1.PrintName(); p2.PrintName()
```

(result)

My name is default

My name is default

8.2 Function Definition in Class

□ Bound and Unbound Methods

- You can call the function in class with two methods;
1) a bound method, and 2) an unbound method.
- The key difference between bound and unbound methods is that an unbound method does not use a particular instance, while a bound method does.

- Ex) 8_02_ClassFunction.py

```
class Person:
    Name = "default"
    def PrintName(self):
        print "My name is " + self.Name
p1 = Person()
p2 = Person()
# a bound method
p1.PrintName(); p2.PrintName()
# an unbound method
Person.PrintName(p1); Person.PrintName(p2)
```

(result)

My name is default

My name is default

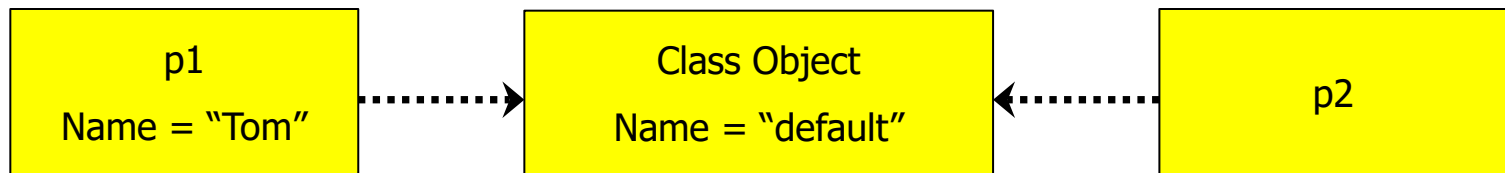
My name is default

My name is default

8.3 Class Namespaces

□ Class Namespace

- When a class is generated, a class object is created. You can allocate the generated class to various instances.
- But, each instance has its own namespace.



- Ex) 8_03_ClassNamespace.py

```
class Person:
    Name = "default"
p1 = Person()
p2 = Person()
print ("Name of p1 instance: " + p1.Name)
print ("Name of p2 instance: " + p2.Name)
p1.Name = "Tom"
print ("Name of p1 instance: " + p1.Name)
print ("Name of p2 instance: " + p2.Name)
```

(result)

Name of p1 instance: default

Name of p2 instance: default

Name of p1 instance: Tom

Name of p2 instance: default

8.3 Class Namespaces

□ **self**

- A method defined in a class body always has a mandatory first parameter, named **'self'**.
- Also you should use **'self'** when you use the variables or functions inside of class.

- Ex) 8_04_ClassSelf.py

```
name = "global"
class Person:
    name = "class"
    def PrintName(self):
        print ("self.name is " + self.name)
        print ("name is " + name)
```

```
p1 = Person()
p1.PrintName()
```

(result)
self.name is class
name is global

8.3 Class Namespaces

□ Alteration of Class

- You can add or remove the variables in the existing class.

- Ex) 8_05_ClassAlteration.py

```
class Person:  
    name = "class"
```

```
Person.year = 1
```

```
p1 = Person()  
p2 = Person()  
p1.year = 3  
print ("Year of p1 instance: {0}".format(p1.year))  
print ("Year of p2 instance: {0}".format(p2.year))
```

```
p2.__class__.year = 2  
print ("Year of p1 instance: {0}".format(p1.year))  
print ("Year of p2 instance: {0}".format(p2.year))
```

(result)

Year of p1 instance: 3

Year of p2 instance: 1

Year of p1 instance: 3

Year of p2 instance: 2

8.4 Inheritance

□ Inheritance

- When you define a class, you can use one or more class objects as optional arguments.

- Ex) 8_06_ClassInheritance.py

```
class Person:
    Name = "parents"
    PhoneNumber = "021-123-4567"

class Student(Person):
    Year = 1

p1 = Student()

print ("Year of p1 instance: {0}".format(p1.Year))
print ("Name of p1 instance: {0}".format(p1.Name))
print ("Phone Number of p1 instance: {0}".format(p1.PhoneNumber))
```

(result)

Year of p1 instance: 1

Name of p1 instance: parents

Phone Number of p1 instance: 021-123-4567